Relational Inductive Biases, Deep Learning, and Graph Networks

Authors: P.W. Battaglia et al.
Talk by Muhammadsharif Nabiev

Moscow Institute of Physics and Technology

2026

The Core Problem

Deep learning has achieved remarkable results:

- Image classification, NLP, game play, continuous control.
- Largely driven by cheap data and cheap compute.

Yet key challenges remain:

- Complex language and scene understanding.
- Reasoning about structured data.
- Transferring learning beyond training conditions.
- Learning from small amounts of experience.

Common thread: all of these demand *combinatorial generalization*.

The tension: end-to-end approaches avoid structure; structured approaches avoid flexibility. *We need both.*

Combinatorial Generalization

Core principle

Constructing new inferences, predictions, and behaviors from known building blocks — "the infinite use of finite means"[1].

How humans achieve it:

- Represent complex systems as *entities* and their *relations*.
- Use hierarchies to abstract away fine-grained differences.
- Compose familiar skills to solve novel problems.
- Draw analogies by aligning relational structure across domains.

Key claim: combinatorial generalization must be a top priority for AI, and *structured representations and computations* are the path toward it.

Approach: incorporate relational inductive biases into deep learning architectures — retain flexibility, gain structure.

---

[1] Humboldt, W. - On Language: On the diversity of human language construction and its influence on the mental development of the human species.

Inductive Biases

Inductive bias: allows a learning algorithm to prioritize one solution over another, independent of observed data. Can be expressed as:

- A prior distribution (Bayesian models).
- A regularization term.
- An architectural constraint.

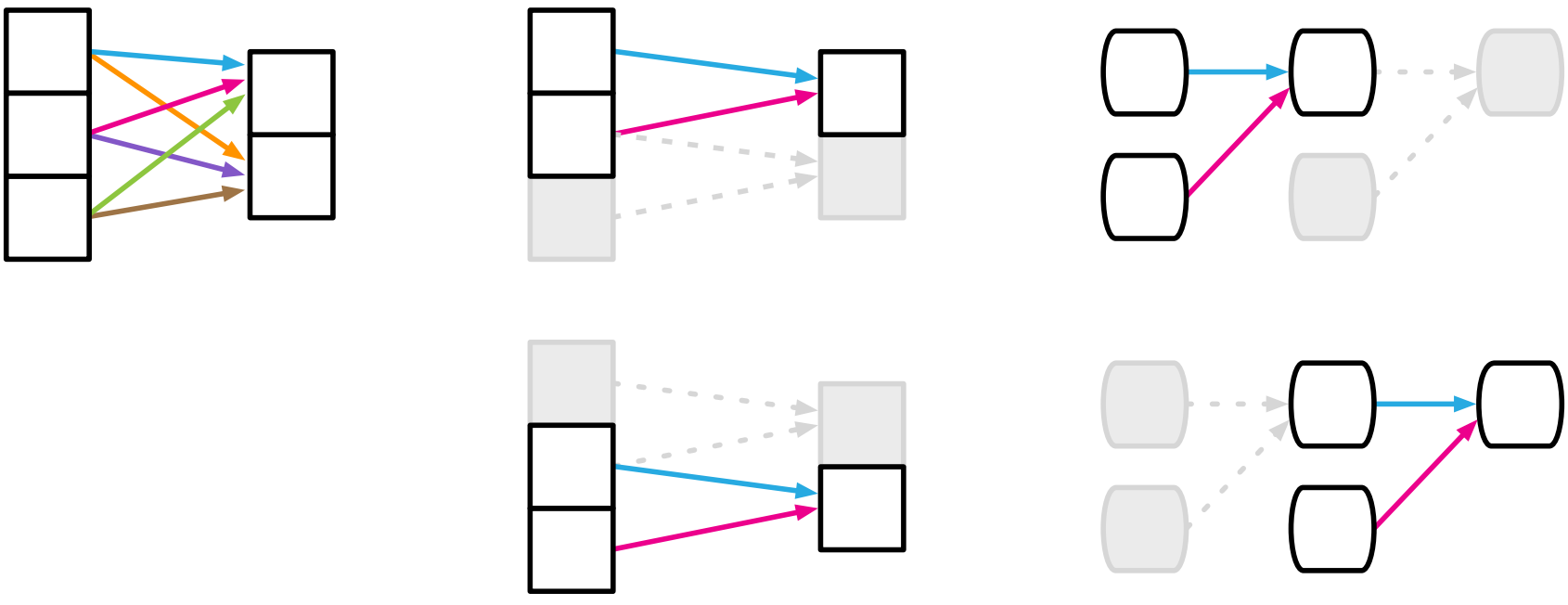
Relational inductive bias: imposes constraints on *relationships and interactions among entities* in a learning process.

Key questions when analyzing a component:

- What are the **entities** and **relations**?
- How is the rule function **reused**?
- Which representations **interact** vs. are **isolated**?

Relational Inductive Biases in Standard Deep Learning

Component	Entities	Relations	Rel. inductive bias	Invariance
Fully connected	Units	All-to-all	Weak	—
Convolutional	Grid elements	Local	Locality	Spatial transl.
Recurrent	Timesteps	Sequential	Sequentiality	Time transl.
Graph network	Nodes	Edges	Arbitrary	Node, edge perm.



(a) (b) (c)

Computations over Sets and Graphs

No standard DL component handles arbitrary relational structure.

Sets encode permutation invariance via symmetric aggregation (Deep Sets):

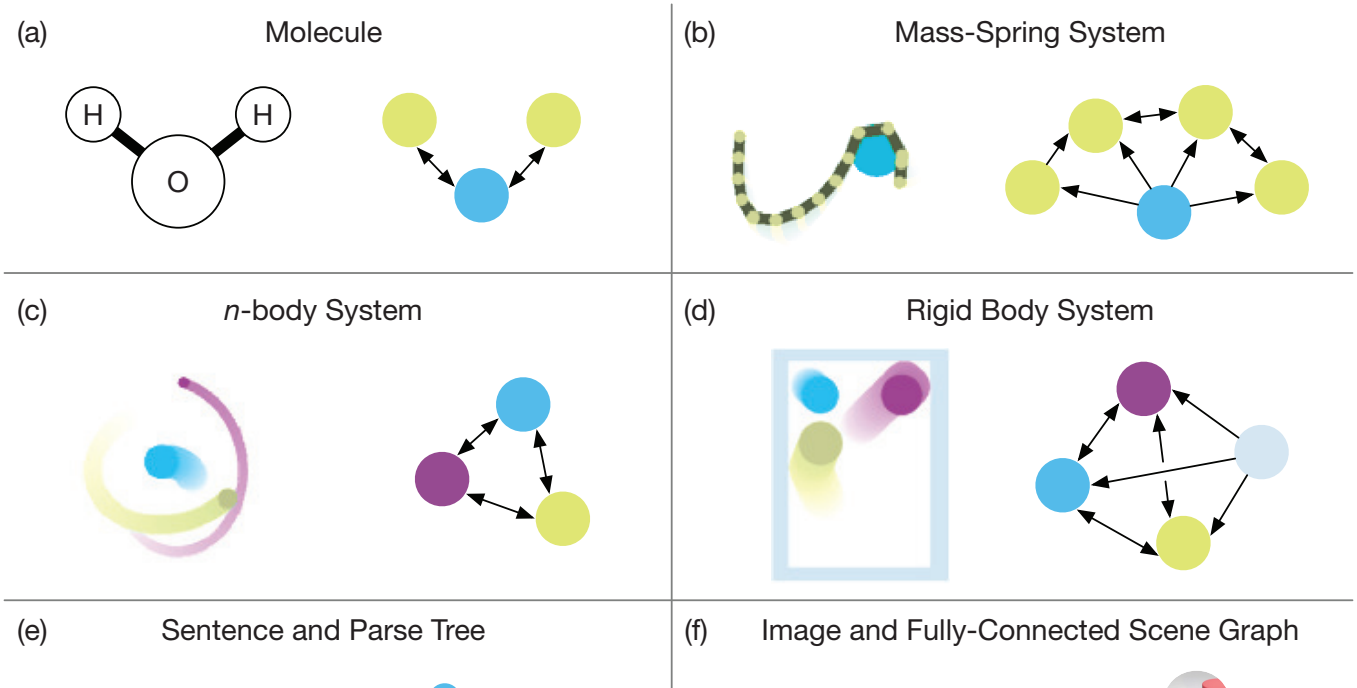
$$x_i' = f\left(x_i, \sum_j g(x_i, x_j)\right)$$

All pairs are considered; the prediction depends only on symmetric functions of the inputs.

Graphs extend this to *sparse* pairwise structure:

$$x_i' = f\left(x_i, \sum_{j \in \delta(i)} g(x_i, x_j)\right)$$

where $\delta(i) \subseteq \{1, \ldots, n\}$ is the neighborhood of node i.



Graph Network (GN) Framework: Definition

A **graph** is a 3-tuple $G = (\mathbf{u}, V, E)$:

- $\mathbf{u}$ — global attribute (e.g. gravitational field).
- $V = \{\mathbf{v}_i\}_{i=1:N^v}$ — node attributes (e.g. position, velocity, mass of each ball).
- $E = \{(\mathbf{e}_k, r_k, s_k)\}_{k=1:N^e}$ — edge attributes with receiver index r_k and sender index s_k (e.g. spring constants).

The **GN block** is a *graph-to-graph* module: takes G as input, performs computations over the structure, returns updated G' as output.

Three design principles

1. Flexible representations.
2. Configurable within-block structure.
3. Composable multi-block architectures.

GN Block: Update and Aggregation Functions

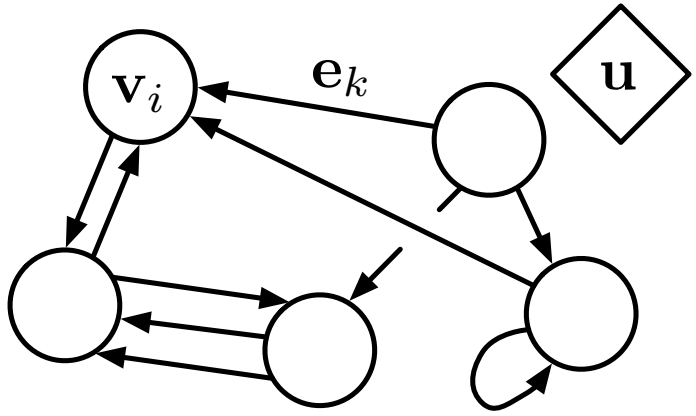
A GN block contains three **update** (ϕ) and three **aggregation** (ρ) functions:

$$\mathbf{e}'_k = \phi^e(\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u})$$
$$\mathbf{v}'_i = \phi^v(\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u})$$
$$\mathbf{u}' = \phi^u(\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u})$$

$$\bar{\mathbf{e}}'_i = \rho^{e\to v}(E'_i), \qquad \bar{\mathbf{e}}' = \rho^{e\to u}(E'), \qquad \bar{\mathbf{v}}' = \rho^{v\to u}(V')$$

Computation order: edge updates $\rightarrow$ node updates $\rightarrow$ global update.

The ρ functions must be *permutation invariant* and accept variable numbers of inputs (e.g. elementwise sum, mean, max).



Full GN Block: Neural Network Implementation

ϕ functions implemented as neural networks (NN^e, NN^v, NN^u):

$$\phi^e(\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u}) := \mathrm{NN}^e([\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u}])$$
$$\phi^v(\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u}) := \mathrm{NN}^v([\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u}])$$
$$\phi^u(\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u}) := \mathrm{NN}^u([\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u}])$$

ρ functions implemented as elementwise summation:

$$\rho^{e\to v}(E'_i) := \sum_{\{k:\, r_k = i\}} \mathbf{e}'_k, \quad \rho^{v\to u}(V') := \sum_i \mathbf{v}'_i, \quad \rho^{e\to u}(E') := \sum_k \mathbf{e}'_k$$

$[\cdot, \cdot]$ denotes concatenation. MLPs are used for vector attributes; CNNs for tensor attributes (e.g. image feature maps). The ϕ functions can also be *RNNs* with an additional hidden state.

GN Variants: Configurable Within-Block Structure

Many published architectures are special cases of the GN framework:

- **MPNN**: ϕ^e omits $\mathbf{u}$; readout R plays role of ϕ^u without taking $\mathbf{u}$ or E' as input.[2]
- **NLNN / Self-Attention**: ϕ^e factored into scalar attention weight $\alpha^e(\mathbf{v}_{r_k}, \mathbf{v}_{s_k})$ and value $\beta^e(\mathbf{v}_{s_k})$; only nodes are output.[3][4]
- **Relation Networks** (Santoro et al., 2017): node update bypassed; global predicted directly from pooled edge representations.[5]
- **Deep Sets**: edge update bypassed; global predicted directly from pooled node representations.[6]

---

[2] Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. (2017). Neural message passing for quantum chemistry.

[3] Wang, X., Girshick, R., Gupta, A., and He, K. Non-local neural networks.

[4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need.

[5] Santoro, A., Raposo, D., Barrett, D. G., Malinowski, M., Pascanu, R., Battaglia, P., and Lillicrap, T. A simple neural network module for relational reasoning.

[6] Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. J. Deep sets.

Composable Multi-Block Architectures

GN blocks compose naturally via their graph-to-graph interface:

$$G' = \mathrm{GN}_2(\mathrm{GN}_1(G))$$

Shared ($\mathrm{GN}_1 = \cdots = \mathrm{GN}_M$): analogous to unrolled RNN / message-passing; information propagates m hops after m steps.
Unshared ($\mathrm{GN}_1 \neq \cdots \neq \mathrm{GN}_M$): analogous to layers of a CNN.

Encode-process-decode (most common design)

1. $\mathrm{GN}_{\mathrm{enc}}$ maps input graph G_{inp} to latent graph G_0.
2. Shared $\mathrm{GN}_{\mathrm{core}}$ is applied M times.
3. $\mathrm{GN}_{\mathrm{dec}}$ maps result to output graph G_{out}.

Relational Inductive Biases in Graph Networks

GNs impose **three strong relational inductive biases:**

1. Arbitrary relational structure.

- Edges define which entities interact; absence of an edge means no direct influence.
- The *input graph* determines interactions, not a fixed architecture.

2. Permutation invariance.

- Entities and relations are represented as sets.
- GN output is invariant to the ordering of nodes and edges.

3. Per-edge and per-node function reuse.

- The same ϕ^e and ϕ^v are applied across *all* edges and nodes.
- A single GN operates on graphs of different sizes (node/edge counts) and shapes (connectivity).
- Directly underpins combinatorial generalization.

Combinatorial Generalization in Graph Networks

Entity- and relation-centric structure directly supports out-of-distribution generalization:

- **Physical simulation** (Battaglia et al., 2016): trained for one-step prediction, GNs simulate thousands of future steps; zero-shot transfer to $2\times/0.5\times$ as many entities.
- **Multi-joint control** (Sanchez-Gonzalez et al., 2018): forward models generalize to agents with unseen numbers of joints.
- **Combinatorial optimization** (Bello et al., 2016; Dai et al., 2017): generalize well to problem sizes far outside the training distribution.
- **Boolean SAT** (Selsam et al., 2018): retains performance across different problem sizes *and* input graph distributions.

Limitations and Open Questions

Known limitations:

- Learned message-passing cannot discriminate all non-isomorphic graphs (Shervashidze et al., 2011).
- Recursion, control flow, and conditional iteration are not natively representable as graphs.

Open questions:

- **Graph construction:** converting raw sensory data (images, text) into a sparse, meaningful graph structure remains unsolved.
- **Adaptive structure:** how to add or remove nodes and edges dynamically during computation (e.g. object fracturing).

Conclusion

What graph networks deliver

A principled framework for relational reasoning that unifies and extends prior work on graph neural networks while combining the strengths of structured approaches and end-to-end deep learning.

The argument:

- Combinatorial generalization is a top priority for AI.
- Standard DL components have limited relational inductive biases.
- GNs impose strong relational structure while remaining fully learnable.

Three design principles:

- Flexible attribute representations (vectors, tensors, graphs).
- Configurable within-block structure (unifies MPNN, NLNN, Relation Nets, Deep Sets).
- Composable multi-block architectures (encode-process-decode, recurrent GN).